

Broadcast Channel

A linear TV channel: one continuously-encoded live HLS stream, and televisions that tune it the way a television tunes a broadcast.

Component	Technology
TV application	LightningJS (Blits 2.10) + hls.js, packaged with Capacitor
Channel server	Node.js, zero dependencies
Encoder	ffmpeg - one process per channel
Transport	HLS with EXT-X-PROGRAM-DATE-TIME
Targets	Android TV today; webOS and Tizen share the same build

What this system does

Several televisions show the same content at the same moment. A television switched on at any time joins whatever is already playing, never from the beginning. After a single setup step, no screen is ever configured again - through reboots, power cuts and app updates.

Contents

- 1 How the system works
- 2 System architecture
- 3 Channels and playlists
- 4 How an MP4 becomes a channel
- 5 The encoder and its supervision
- 6 Keeping televisions in sync
- 7 Pairing a television
- 8 Screen wireframes
- 9 API reference
- 10 Running the system
- 11 Operating it
- 12 Known limitations

SECTION 1

How the system works

The server behaves like a television station. It plays a playlist of videos continuously, around the clock, whether any television is switched on or not. Televisions do not request content, schedule anything, or coordinate with each other - they simply display whatever is currently being broadcast.

The three ideas behind it

Idea	Consequence
The stream is produced once, centrally	What is on air is decided in exactly one place. There is no agreement to maintain between devices, because there is nothing for them to disagree about.
Video is delivered as ordinary static files	No sessions, no handshakes, no per-viewer state. The origin does not know or care how many televisions exist, which is what allows a CDN to absorb any number of them.
Televisions hold a fixed delay behind a shared clock	Every screen targets the same instant, so independent devices that never communicate still land on the same frame.

Why the stream is continuous

A file has a beginning, so opening one starts at zero. A broadcast has no beginning - you join whatever is happening now. Producing the stream continuously is what turns playback position from something each television must calculate into something it simply receives.

This is also why clip transitions are invisible. The encoder treats the whole playlist as one endless input, so moving from one video to the next is not an event on the television at all - it is just the next four seconds of the same stream.

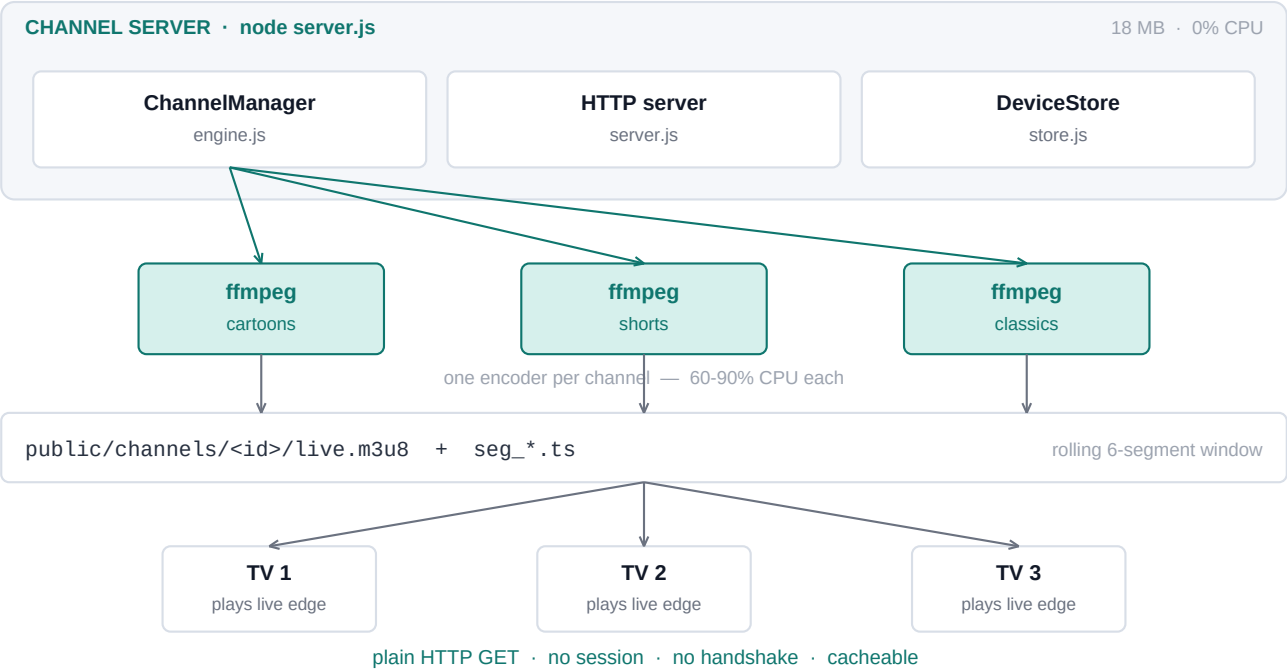
The one thing to internalise

Cost scales with the number of channels, not the number of viewers. A thousand televisions watching one channel is the same work as one television watching it. Adding a second channel costs an entire additional encoder.

SECTION 2

System architecture

One Node process supervises everything and serves files. It never touches video data - that is entirely ffmpeg's job.



The measurement that explains the design

On a running system, Node uses **0% CPU and 18 MB** of memory, while the three ffmpeg processes use **60-90% of a core each**. Everything expensive happens inside ffmpeg.

Adding...	Cost
1,000 more televisions	Almost nothing. Static file reads, and a CDN absorbs them entirely.
One more channel	A whole additional encoder - 60-90% of a CPU core.

Three independent concerns

Concern	File	Responsibility
Encoding	engine.js	Spawns and supervises one ffmpeg per channel. No HTTP at all.
Serving	server.js	All APIs, static file serving, and cache policy. No video logic.
State	store.js	Device records: pairing codes, tokens, last-seen timestamps.

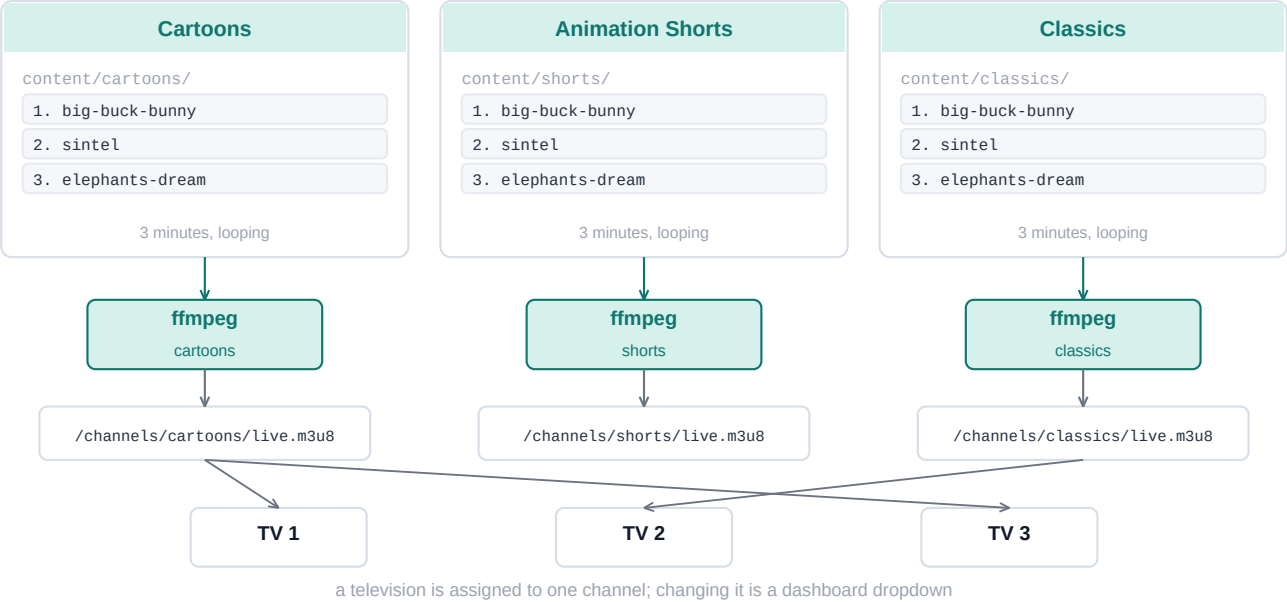
The video path depends on nothing

If the device store and the API layer were both deleted, every already-paired television would keep playing indefinitely - playback is nothing more than repeated GETs of static files. Losing the control plane costs you the dashboard, not the picture.

SECTION 3

Channels and playlists

A channel is a directory of video clips plus one encoder process. Three exist in this system, each with three videos.



Adding a channel

Create a directory of clips under content/, write a playlist.txt concat list, and add an entry to channels.json. The server spawns an encoder for it on next start.

```
{
  "channels": [
    { "id": "cartoons", "name": "Cartoons" },
    { "id": "shorts", "name": "Animation Shorts" },
    { "id": "classics", "name": "Classics" }
  ]
}
```

Source clips must match exactly

Every clip within a channel must share identical codec, resolution, frame rate and audio layout. The concat demuxer joins them without renegotiating, so a mismatch is the usual reason a channel dies at a clip boundary.

Parameter	Value used here
Video	H.264 (libx264), main profile, yuv420p
Resolution	1280 x 720
Frame rate	25 fps, keyframe every 2 seconds
Bitrate	2000 kbps, capped
Audio	AAC, 128 kbps, 48 kHz stereo

The preparation script normalises every source to these values regardless of what it started as, which is what makes arbitrary input files safe to drop in.

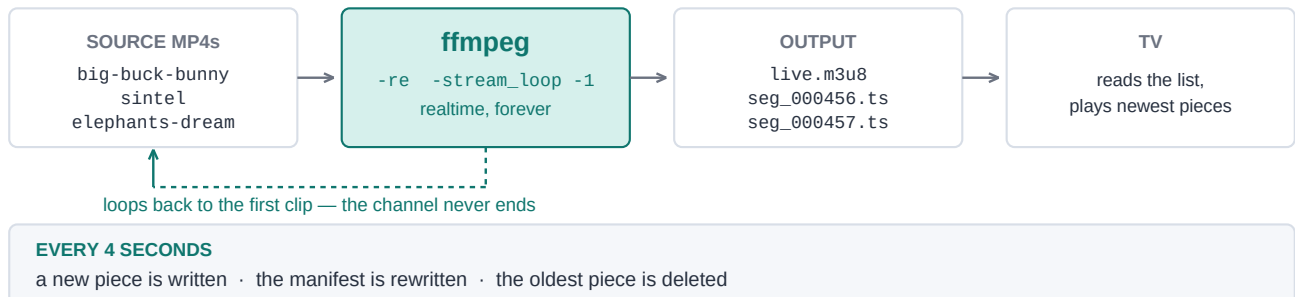
Assigning a television to a channel

Each paired television carries a channel assignment. Changing it is a dropdown in the dashboard. Nothing is pushed to the television - it discovers the new channel URL on its next heartbeat, within 30 seconds, and reattaches itself.

SECTION 4

How an MP4 becomes a channel

A television cannot join the middle of a file. So ffmpeg continuously chops the source video into small pieces and maintains a short list of the newest ones.



This is not ordinary conversion

Normal conversion runs once and produces a complete set of files. This runs **at realtime speed, forever**, and keeps only the newest handful of pieces.

Observed on a running channel

Time on air	30 minutes
Pieces created	457 (seg_000000 -> seg_000456)
Pieces still on disk	7
Disk used by all three channels	23 MB
Source MP4s	never modified - read again and again

The command

```
ffmpeg -re -stream_loop -1 -f concat -safe 0 -i playlist.txt \
-c:v libx264 -g 50 -c:a aac \
-f hls -hls_time 4 -hls_list_size 6 \
-hls_flags delete_segments+append_list+program_date_time \
public/channels/cartoons/live.m3u8
```

Flag	Why it matters
-re	Paces reading at realtime. Without it ffmpeg races through the files and nothing is live.
-stream_loop -1	Loops the playlist forever - the channel never ends.
-f concat	Treats the playlist as one continuous input, which is why clip transitions are invisible.
-hls_time 4	Four-second pieces; -g 50 places a keyframe every two seconds so pieces always split cleanly.
-hls_list_size 6	A rolling window of roughly 24 seconds.
delete_segments	Stops the disk filling over days of uptime.
append_list	Survives a restart without resetting the media sequence.

`program_date_time` Stamps every piece with a wall-clock time. This is what makes frame-accurate sync possible.

Why re-encode instead of -c copy

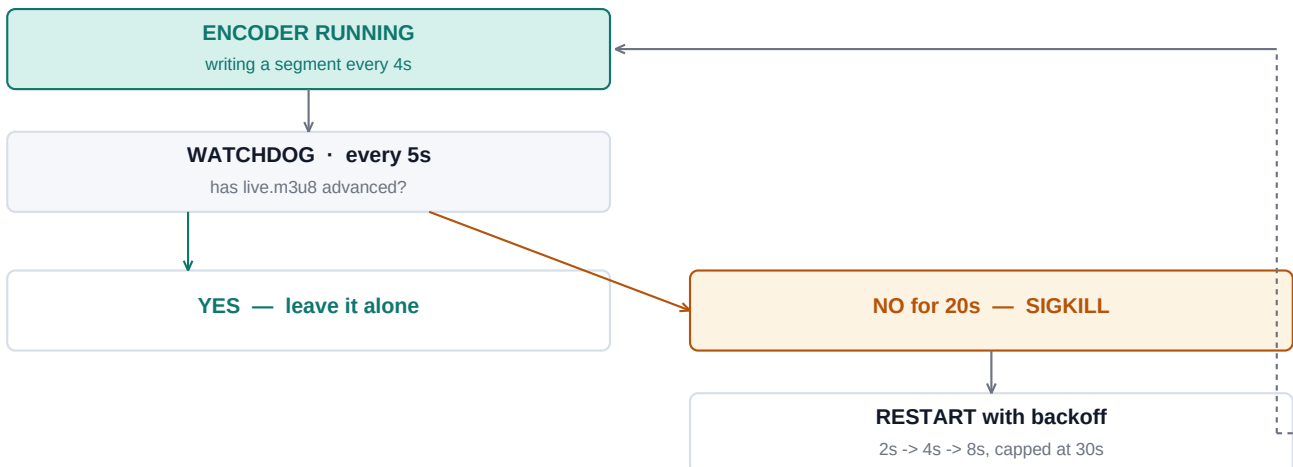
Copying is far cheaper and is the right choice once all source files share identical parameters. But looping a concatenated playlist with -c copy produces timestamp discontinuities at every wrap, which some players show as a stutter. Re-encoding regenerates clean timestamps. Switching is a one-word change once sources are known-identical, and drops CPU use to almost nothing.

SECTION 5

The encoder and its supervision

Encoders die. A malformed source file, a disk hiccup, an out-of-memory kill. A channel without a restart policy is a channel that goes dark at three in the morning.

The harder failure is subtler: an encoder that **stops producing output while staying alive**. It has not crashed, has not exited, and passes every conventional health check - while every viewer sees a frozen screen.



WHY SEGMENT PRODUCTION, NOT PROCESS LIVENESS

An ffmpeg that wedges stays alive and passes every "is it running?" check while producing nothing.

This is not hypothetical

During testing, one encoder ran normally for eighteen minutes and then stopped writing segments while remaining alive. The watchdog noticed the manifest had not advanced, killed the process, and restarted it. The channel recovered without intervention.

```
[cartoons] manifest has not advanced in 20s - killing wedged ffmpeg
[cartoons] ffmpeg exited code=null signal=SIGKILL after 1086s
[cartoons] restarting in 2000ms (restart #1)
```

Backoff behaviour

Restart delay doubles on each consecutive failure up to a 30-second ceiling, then resets to two seconds once a process has survived a full minute. A single transient failure therefore does not leave the channel waiting half a minute to recover, while a genuinely broken channel does not spin in a tight restart loop.

What a restart costs viewers

Televisions on that channel rebuffer for a segment or two and then continue. Because the restart appends to the existing manifest rather than resetting the media sequence, players recover on their own without being reloaded.

SECTION 6

Keeping televisions in sync

Every television pulling the same manifest guarantees they show the same **content**. It does not put them on the same **frame**. Each player picks its own start point near the live edge, and once two televisions drift apart nothing pulls them back - live HLS has no restoring force.

So each television holds a fixed latency behind a shared clock:

```
target = serverNow() - 12000ms
drift = hls.playingDate - target          // playingDate comes from PDT
```

|drift| > 5000 ms

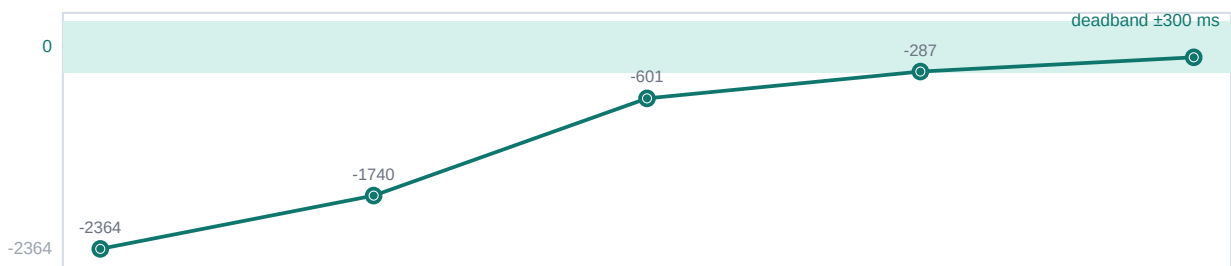
seek

300 - 5000 ms

playbackRate 0.98 / 1.02

<= 300 ms

leave it alone



measured on an Android TV emulator: drift closes at exactly the 2% that 1.02x predicts

The server clock

`serverNow()` is round-trip corrected: `t0` is recorded **before** the request and the offset is taken from the midpoint, sampled several times with the lowest-latency result kept. Measuring after the response arrives bakes in half the round trip and biases every television differently.

Why this matters more than it sounds

During testing an Android emulator had a clock 10.5 minutes out from the host. Without correction that television would have shown content ten minutes away from every other screen. The correction absorbed it silently, and the two clients still agreed on frame.

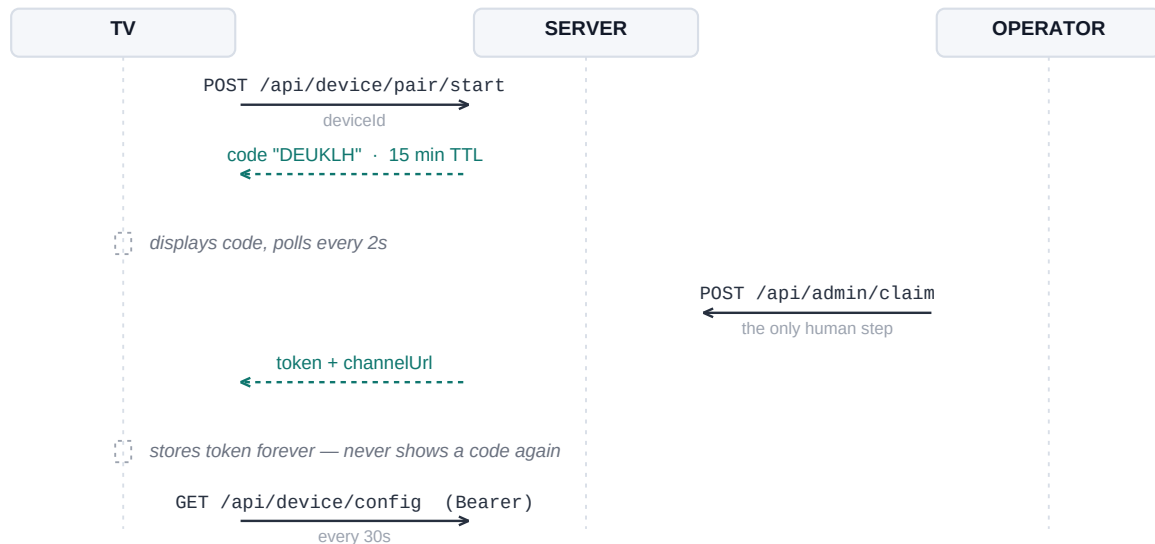
Why the rate correction is gentle

Corrections use plus or minus two percent. Larger adjustments close the gap faster but are audible as a pitch shift on anything with sound, and visible as judder. Two percent closes a two-second error in roughly a hundred seconds, which is imperceptible and still fast enough in practice.

SECTION 7

Pairing a television

A television is an appliance, not a personal device. It is authorised once by an operator and never asks anyone for anything again.



The code is a doorbell, not a key

The six-character code grants nothing. It exists only so a human can point at one specific television among many. The real credential is the token, which the server creates **after** approval.

Property	Value	Reason
Alphabet	A-Z and 2-9, minus I, O, 0, 1	Read off a screen across a room and typed elsewhere
Length	6 characters	$32^6 \sim 1.07$ billion combinations
Lifetime	15 minutes	Long enough for setup, short enough to expire
Generated with	<code>crypto.randomInt</code>	<code>Math.random</code> is not suitable for credentials
Token	48 hex characters	Issued only on approval; stored permanently

Why not a login screen

Consider a building-wide power cut at 6am. With a login screen, somebody walks to every television and types an email address using a remote control on an on-screen keyboard. With pairing, every screen returns on its own. This single consideration drives the entire design.

Unpairing

Removing a device deletes its record and token. The television is not notified - nothing is ever pushed to a television. Within 30 seconds its heartbeat returns 401, at which point it clears its stored token and displays a fresh pairing code.

Knowing a television is online

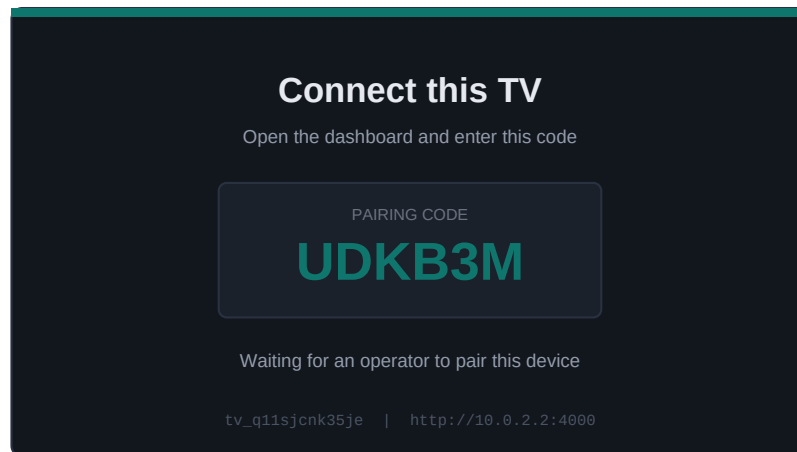
There is no connection to monitor, so the television reports in. Its 30-second heartbeat stamps a last-seen time, and the dashboard marks a screen online if it has been seen within 90 seconds - three missed beats. The heartbeat is not on the video path: if it fails entirely, playback continues and the television merely goes grey in the dashboard.

SECTION 8

Screen wireframes

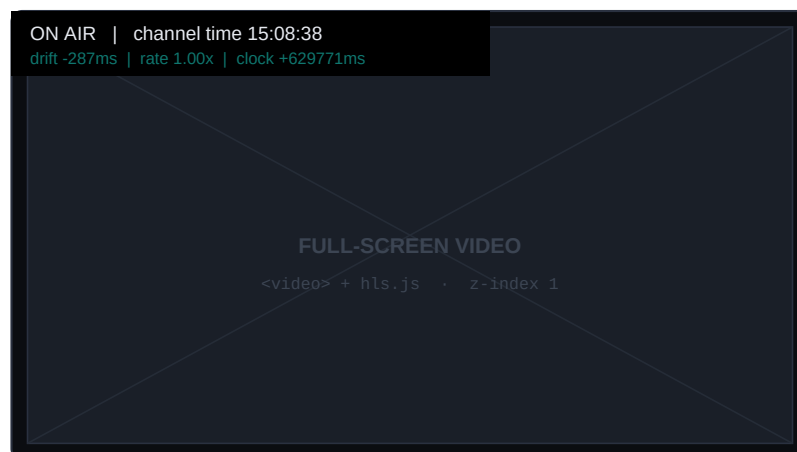
Three screens exist in the entire system. Most televisions only ever display the second one.

8.1 Television - pairing (first boot only)



Shown once per TV, ever. After pairing this screen is never seen again — not on reboot, not after a power cut, not after an app update.

8.2 Television - player (every boot thereafter)



Lightning canvas is layered ABOVE the video (z-index 2) so overlays draw on top. It must stay transparent — an opaque root hides the picture while text still shows.

8.3 Operator dashboard (browser)

Channel Control

Each channel is its own continuously-encoded HLS stream.

CHANNELS

● Cartoons	3 videos	live · 0 restarts
● Animation Shorts	3 videos	live · 0 restarts
● Classics	3 videos	live · 0 restarts

PAIR A TV

Pair device

DEVICES

NAME	STATUS	PLAYING	
TV 1	● online	Cartoons v	Remove
TV 2	● online	Classics v	Remove
TV 3	● waiting · A3CK47	— v	Remove

The dashboard is the only place a television can be authorised, moved between channels, or removed. Channel health, uptime and restart counts are shown per channel, and each paired television carries a dropdown that reassigns it to a different playlist. The change reaches the television on its next heartbeat, within 30 seconds.

SECTION 9

API reference

Every endpoint lives in one file, `channel-server/server.js`, matched by a plain "METHOD /path" string - no framework, no router library.

Device endpoints

Endpoint	Purpose
POST <code>/api/device/pair/start</code>	TV requests a pairing code. Unauthenticated.
GET <code>/api/device/pair/status</code>	TV polls every 2s until an operator claims it.
GET <code>/api/device/config</code>	Boot config and 30s heartbeat. Bearer token. The only authenticated call.

Operator endpoints

Endpoint	Purpose
POST <code>/api/admin/claim</code>	Authorise a device onto a channel. The system's trust boundary.
POST <code>/api/admin/unpair</code>	Delete a device and its token.
POST <code>/api/admin/device/channel</code>	Move a television to another channel.
GET <code>/api/admin/devices</code>	Fleet list with online state.
GET <code>/api/admin/channels</code>	Channel list with health.
POST <code>/api/admin/reload</code>	Restart one encoder after a playlist change.
GET <code>/api/status</code>	All channel health plus server time.

The video path

Endpoint	Purpose
GET <code>/channels/<id>/live.m3u8</code>	The channel. No token, no session, no headers.
GET <code>/channels/<id>/seg_*.ts</code>	Media segments. Immutable once written.

Cache policy

File	Header	Reason
<code>*.m3u8</code>	<code>no-cache, no-store</code>	Rewritten every four seconds
<code>*.ts</code>	<code>max-age=31536000, immutable</code>	Never changes once written

Cache headers decide whether this works

A CDN caching the manifest stalls every television at once, and looks exactly like an encoder failure while you debug it.

SECTION 10

Running the system

Prepare content, once

```
cd channel-server && ./fetch-content.sh
```

Downloads 60-second excerpts from three Blender open movies and normalises them to identical encoding parameters.

Start the channels

```
cd channel-server && node server.js
```

Starts one ffmpeg per channel and serves the dashboard on port 4000. Stop with Ctrl+C; the server shuts its encoders down cleanly rather than orphaning them.

Build and install the television app

```
cd tv-app && npm install
npm run android:apk
adb install -r android/app/build/outputs/apk/debug/app-debug.apk
```

Project layout

Path	Contents
channel-server/server.js	All APIs, static serving, cache policy
channel-server/engine.js	ffmpeg supervision, one engine per channel
channel-server/store.js	Device records: codes, tokens, last-seen
channel-server/channels.json	Channel definitions
channel-server/fetch-content.sh	Downloads and normalises source clips
channel-server/public/admin.html	Operator dashboard
tv-app/src/pages/Pairings.js	First-boot pairing screen
tv-app/src/pages/Player.js	The entire client: player and latency hold
tv-app/src/utils/device.js	Identity, credentials, server clock

Porting to webOS and Tizen

The television app builds to three plain web files. Nothing platform-specific lives in the player - it is a video element, hls.js and a drift loop. The Android-only pieces are the Capacitor wrapper, MainActivity, and one branch inside getApiBase(). Adding a platform means adding a packaging directory and a branch in that one function.

Two LightningJS details that cost real debugging time

Dynamic bindings need the colon prefix - `content="$code"` renders once and never updates, while `:content="$code"` is reactive. And nothing layered above the video may be painted opaque: the canvas sits above the video element, so an opaque root hides the picture while overlay text still draws, which looks exactly like playback failure.

SECTION 11

Operating it

What to watch

The dashboard surfaces everything that matters, and the same data is available from `GET /api/status` for external monitoring.

Signal	Healthy	What it means if not
onAir	true	The manifest has not been written in the last 15 seconds.
manifestAgeSeconds	under 5	Segment production is falling behind or has stopped.
restartCount	stable	A climbing count means an encoder is failing repeatedly.
segmentsOnDisk	6 to 8	Far more suggests deletion is failing and the disk will fill.
device online	true	No heartbeat for 90 seconds. The screen may still be playing.

Changing what is on air

Task	How	Effect on viewers
Move a TV to another channel	Dashboard dropdown	That screen switches within 30 seconds. Others unaffected.
Add or remove a video	Edit the directory and playlist, then reload that channel	Televisions on that channel rebuffer briefly. Other channels unaffected.
Add a channel	New content directory plus a channels.json entry, then restart	All channels restart. Plan for a quiet period.

Capacity planning

Encoding is the only meaningful cost: each channel consumes most of one CPU core. Viewers cost almost nothing, because every television is served the same static files. If encoder CPU becomes the constraint, normalise all source clips to identical parameters and switch the engine to stream copying, which removes the encode entirely.

Recovering from problems

Symptom	Likely cause	Action
One channel frozen, others fine	Wedged encoder	The watchdog restarts it within 20 seconds. A rising restart count means the source files need checking.
All televisions stall at once	Manifest being cached	Check cache headers at every hop. This almost always means a proxy or CDN.
A television shows a pairing code unexpectedly	Its record was removed	Re-pair it from the dashboard.
A television is stuck reconnecting	Server unreachable at boot	It retries indefinitely and recovers on its own once the server returns.

SECTION 12

Known limitations

This is a proof of concept. The architecture is sound; the following must be addressed before production use.

#	Limitation	Impact
1	Operator APIs have no authentication	Anyone who can reach the port can remove televisions or reassign channels. Fix this first.
2	Playlist changes restart the encoder	The concat demuxer reads its list once, so changing content is a visible cut on air for that channel.
3	Device registry is a JSON file	Cannot be shared between instances: no horizontal scaling, and a deploy drops pairing state.
4	One origin, no CDN	Every television pulls segments from a single Node process.
5	No alerting	The supervisor recovers a wedged encoder silently. If it failed to restart, nobody would be paged.
6	Audio is muted in the player	A television has no user gesture to unlock audio. The Android WebView flag is already set, so removing one line should suffice.
7	No HTTPS	Fine on a controlled network, unacceptable across the internet.

Verified behaviour

The following were observed on a running system rather than assumed.

Behaviour	Evidence
Drift correction converges	Drift closed from -2364 ms to within the 300 ms deadband, at exactly the 2% that 1.02x predicts.
Clock skew is absorbed	A client whose device clock was 10.5 minutes out still agreed on frame with a correctly-set one.
Two platforms stay together	An Android TV and a browser client showed the same frame with no communication between them.
Wedged encoders are caught	An ffmpeg process stopped producing output while still alive; the watchdog detected the stalled manifest and restarted it.
Late joiners land correctly	A television started mid-stream joined the live edge rather than the beginning.
Clip transitions are seamless	A channel moved between videos with no gap and no action on any television.

Source: github.com/shashikant8383/VideoSync